

Ex. No. : 09

Date:

Register No : 231701007

Name: Badrish R S

A python program to implement KNN and K-means

Aim:

To implement a python program that implement KNN and K-means

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
import numpy as np from
collections import Counter import
matplotlib.pyplot as plt
```

```
class KNearestNeighbors: def
```

```
__init__(self, k=3):
```

```
    self.k = k
```

```
    def fit(self, X,
```

```
    y):
```

```
        self.X_train = X        self.y_train
```

```
        = y
```

```

def _euclidean_distance(self, x1, x2):
    return np.sqrt(np.sum((x1 - x2)**2))

def predict(self, X):
    predictions = [self._predict(x) for x in X]
    return np.array(predictions)

def _predict(self, x):
    distances = [self._euclidean_distance(x, x_train) for x_train in self.X_train]

    k_nearest_indices = np.argsort(distances)[:self.k]
    k_nearest_labels = [self.y_train[i] for i in k_nearest_indices]

    most_common = Counter(k_nearest_labels).most_common(1)
    return most_common[0][0]

class KMeans:
    def __init__(self, n_clusters=3, max_iter=100,
                 random_state=None):
        self.n_clusters = n_clusters

        self.max_iter = max_iter
        self.random_state = random_state
        self.centroids = None

        self.labels = None

```

```

def _initialize_centroids(self, X):
    np.random.seed(self.random_state)    indices =
    np.random.choice(X.shape[0], self.n_clusters, replace=False)    return
    X[indices]

```

```

def _assign_clusters(self, X, centroids):
    distances = np.array([
        [np.linalg.norm(x - centroid) for centroid in centroids]    for
x in X
    ])
    return np.argmin(distances, axis=1)

```

```

def _update_centroids(self, X, labels):
    new_centroids = np.zeros((self.n_clusters, X.shape[1]))    for
i in range(self.n_clusters):
        points_in_cluster = X[labels == i]    if
len(points_in_cluster) > 0:
            new_centroids[i] = np.mean(points_in_cluster, axis=0)
    return new_centroids

```

```

def fit(self, X):
    self.centroids = self._initialize_centroids(X)

```

```

        for _ in range(self.max_iter):

            self.labels = self._assign_clusters(X, self.centroids)
new_centroids = self._update_centroids(X, self.labels)

            if np.allclose(self.centroids, new_centroids):

                break        self.centroids =

new_centroids

            return self.labels, self.centroids

def predict(self, X):        if self.centroids

is None:

            raise RuntimeError("KMeans must be fitted before predicting.")

return self._assign_clusters(X, self.centroids)

if __name__ == "__main__":

    from sklearn.datasets import make_blobs, make_classification    from

sklearn.model_selection import train_test_split    from sklearn.metrics

import accuracy_score

    print("\n--- K-Nearest Neighbors (KNN) Example ---")

```

```
X_cls, y_cls = make_classification(n_samples=200, n_features=2, n_informative=2,  
n_redundant=0, n_clusters_per_class=1, random_state=42)
```

```
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls,  
test_size=0.3, random_state=42)
```

```
knn = KNearestNeighbors(k=5)  
  
knn.fit(X_train_cls, y_train_cls)    y_pred_knn  
  
= knn.predict(X_test_cls)
```

```
accuracy_knn = accuracy_score(y_test_cls, y_pred_knn)  
  
print(f'KNN Accuracy: {accuracy_knn * 100:.2f}%')
```

```
try:
```

```
    plt.figure(figsize=(10, 5))    plt.subplot(1, 2, 1)    x_min,  
x_max = X_cls[:, 0].min() - 0.5, X_cls[:, 0].max() + 0.5    y_min,  
y_max = X_cls[:, 1].min() - 0.5, X_cls[:, 1].max() + 0.5    xx, yy =  
np.meshgrid(np.linspace(x_min, x_max, 50),  
np.linspace(y_min, y_max, 50))
```

```
    Z_knn = knn.predict(np.c_[xx.ravel(), yy.ravel()])
```

```
    Z_knn = Z_knn.reshape(xx.shape)
```

```
    plt.contourf(xx, yy, Z_knn, alpha=0.3, cmap=plt.cm.RdBu)    plt.scatter(X_cls[:,  
0], X_cls[:, 1], c=y_cls, s=20, edgecolor='k', cmap=plt.cm.RdBu)    plt.title('KNN
```

```
Decision Boundary (k=5')      plt.xlabel('Feature 1')      plt.ylabel('Feature 2')
```

```
except ImportError:
```

```
    print("Matplotlib not installed. Skipping KNN plot.")
```

```
print("\n--- K-Means Clustering Example ---")
```

```
        X_km,          y_true_km      =      make_blobs(n_samples=300, centers=4,
cluster_std=0.8, random_state=42)  n_clusters_km = 4
```

```
kmeans = KMeans(n_clusters=n_clusters_km, random_state=42)
```

```
labels_km, centroids_km = kmeans.fit(X_km)
```

```
print(f"K-Means performed with {n_clusters_km} clusters.")
```

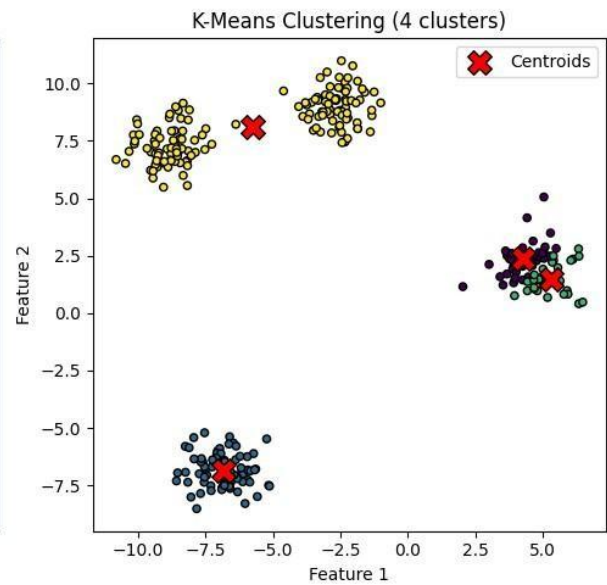
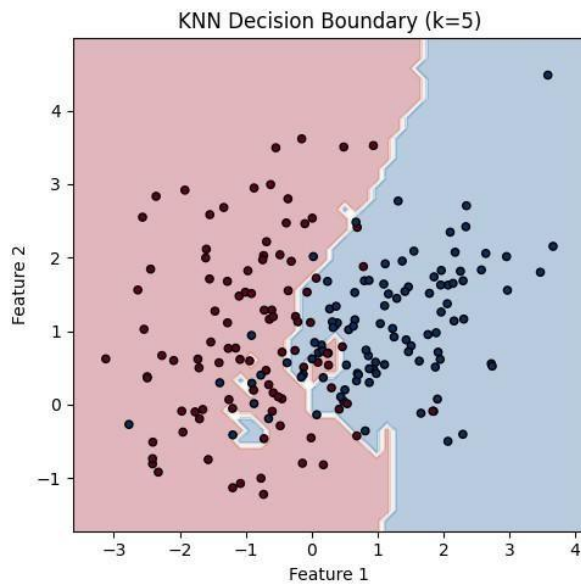
```
print(f"Final Centroids:\n{centroids_km}")
```

```
try:
```

```
    plt.subplot(1, 2, 2)      plt.scatter(X_km[:, 0], X_km[:, 1], c=labels_km, s=20,
cmap='viridis', edgecolor='k')      plt.scatter(centroids_km[:, 0], centroids_km[:, 1], marker='X',
s=200, c='red',              edgecolor='black', label='Centroids')      plt.title(f"K-Means
Clustering ({n_clusters_km} clusters)")      plt.xlabel('Feature 1')      plt.ylabel('Feature 2')
plt.legend()      plt.tight_layout()      plt.show()
```

```
except ImportError:
```

```
    print("Matplotlib not installed. Skipping K-Means plot.")
```



Output:

--- K-Nearest Neighbors (KNN) Example ---

KNN Accuracy: 80.00%

--- K-Means Clustering Example --- K-Means performed

with 4 clusters.

Final Centroids:

[[4.30482919 2.37577687]

[-6.84180708 -6.84038791]

[5.33339082 1.47653843]

[-5.73523256 8.10177082]]

Result:

The python program has been successfully implemented